Programmation sur Processeur Graphique

—

Part 0.5: Rappels (rapides) en C

Pierre-Emmanuel Hladik
pierre-emmanuel.hladik@ec-nantes.fr

22 janvier 2025

Objectifs

- Rappel de la syntaxe du C
- Rappel sur la compilation en C
- Focus sur la libC et la gestion mémoire

Rappel : éléments de syntaxe de base

```c
#include <stdio.h>

int main(void)
{
    printf("hello, world\n");
    return 0;
}
```

- `#include <stdio.h>` inclut le fichier d'en-tête `<stdio.h>` contenant les déclarations des fonctions d'entrées-sorties de la libC, dont `printf`
- `main` est le nom de la fonction principale, aussi appelée point d'entrée du programme. `int` est le type (entier) renvoyé par la fonction `main`
- Les parenthèses `()` après main indiquent qu'il s'agit d'une fonction. Le mot-clé `void` signifie que la fonction n'a aucun paramètre.
- Les accolades `{` et `}` entourent les instructions constituant le corps de la fonction.
- `printf` est une fonction d'écriture dans la sortie standard, qui produit l'affichage dans la console par défaut. Les caractères `"` délimitent une chaîne de caractères et les caractères `\n` sont une séquence d'échappement représentant le saut de ligne.
- Un point-virgule `;` termine l'instruction.
- L'instruction `return 0;` indique que la fonction `main` retourne la valeur 0. Cette valeur est de type `int`, et correspond au `int` devant le `main`.

Eléments lexicaux de base

Les mots réservés

C89 32 mots réservés : `auto`, `break`, `case`, `char`, `const`, `continue`, `default`, `do`, `double`, `else`, `enum`, `extern`, `float`, `for`, `goto`, `if`, `int`, `long`, `register`, `return`, `short`, `signed`, `sizeof`, `static`, `struct`, `switch`, `typedef`, `union`, `unsigned`, `void`, `volatile`, `while`

C99 5 mots réservés : `_Bool`, `_Complex`, `_Imaginary`, `inline`, `restrict`

C11 7 mots réservés : `_Alignas`, `_Alignof`, `_Atomic`, `_Generic`, `_Noreturn`, `_Static_assert`, `_Thread_local`,

C23 à venir (14 nouveaux mots)

Focus : sizeof

Syntaxe : `sizeof ( type-name )`
Retourne le nombre d'octets pour stocker un objet du type de l'opérande.

Exemple : `sizeof(double)`

Les types numériques en C

Entiers

- Type signé par défaut (`signed`) (complément à deux)
- Version non signée (`unsigned`) (pas de signe)
- Les types existants sont : `char` (8 bits), `short` (16 bits min), `int` (16 bits min), `long` (32 bits min), `long long` (64 bits min)

Exemple :

Type	Bits	Minimum	Maximum
`short`	16	-32768	32767
`unsigned int`	32	0	4294967295

Flottants

- représentation suivant le standard IEEE-754 en simple ou double précision
- Les types existants sont : `float` (4 octets), `double` (8 octets), `long double` (10 octects)

Expression conditionnelle

```
if (x > 10)
{
        my_variable = "foo";
}
else
{
        my_variable = "bar";
}
```

Remarque sur les accolades

Pas obligatoires mais à vos risques et périls...

```
if (x > 10)
        my_variable = "foo";
else
        my_variable = "bar";
```

Opérateur ternaire

```
condition ? evaluated-when-true : evaluated-when-false
```

Exemple :

```
my_variable = x > 10 ? "foo" : "bar";  // egal "foo" si x > 10 et "bar" sinon
```

Boucles

Boucle While

Syntaxe : `while ( expression )statement`

Exemple :

```
int x = 0;
while (x < 5) {
        printf ("x = %d\n", x);
        x++;
}
```

Boucle For

Syntaxe : `for (init-expression; cond-expression; loop-expression)statement`

Exemple :

```
for (int i = 0; i < 100; i++) {
    for (int j = i; j < 10; j++) {
                some_function(i, j);
    }
}
```

Déclaration d'un pointeur

Un pointeur est un type contenant une adresse mémoire (i.e. une référence).
Syntaxe : `type * nom_du_pointeur;`

Exemple :

```
int *ptr;
```

Pointeur NULL

Le pointeur NULL est le pointeur qui ne pointe sur aucun autre emplacement que NULL.
Syntaxe : `type pointer_name = NULL;` ou `type pointer_name = 0;`

Exemple :

```
int* ptr = NULL;
```

Pointeurs : référencement et déréférencement

Opérateur d'adressage

L'opérateur & permet d'obtenir l'adresse d'un objet (sa référence).

Exemple :

```
int a = 10;
int *p = &a;
```

Déférencement

L'accès au contenu d'une adresse se fait avec l'opérateur d'indirection * (ou de déréférencement).

Exemple :

```
int a = 10;
int *p = &a;
*p = 20;
printf("a = %d\n", a); // affiche "a = 20"
```

Manipuler les pointeurs

Incrémenter (ou décrémenter) un pointeur

Il est possible d'incrémenter (ou décrémenter) un pointeur pour pointer sur une référence décalée de l'incrément (ou du décrément). Le décalage se fait en fonction de la taille du type pointé.

Exemple :

```
int a = 10;
int * prt = &a;
double b = 10;
double * prt2 = &b;

ptr  = ptr + 2; // decalage de 2*sizeof(int)
ptr2  = ptr2 + 3; // decalage de 3*sizeof(double)
```

Pointeur générique

Pointeur de type void

Le type `void *` est un pointeur générique, il référence n'importe quel type de pointeur. Il est possible d'affecter n'importe quelle adresse à un pointeur `void *` et d'affecter un pointeur `void *` à n'importe quel autre pointeur (et inversement).

Exemple :

```
int a;
double b;
void *p;
double *r;

p = &a; /* Correct */
p = &b; /* Correct */
r = p; /* Correct */
```

Affichage d'une adresse

Il est possible d'afficher une adresse à l'aide de l'indicateur de conversion %p de la fonction `printf()`. Ce dernier attend en argument un pointeur de type `void *`.

Exemple :

```
int a;
int *p = &a;
printf("%p == %p\n", (void *)&a, (void *)p);
```

Pointeur de pointeur

Un pointeur a lui aussi une adresse. Il est donc possible de créer un pointeur qui pointe sur ce pointeur.
Exemple :

```
int a = 10;
int *pa = &a;
int **pp = &pa;
```

Pointeur de fonction

Un pointeur de fonction est un pointeur sur la première instruction d'une fonction.
Syntaxe : `type (*identificateur)(parametres);`
Exemple :

```
void (*ptr_fonction) (int,char); // pointeur sur une fonction qui a 2 arguments du type (int, char)
      et ne retourne rien.

void affiche(int) {}
void (*ptrf) (int);
ptrf = &affiche; // le pointeur prend l'adresse de la fonction affiche
(*ptrf)(3); // appel de la fonction pointee par ptrf
```

Bilan sur les pointeurs

- Un pointeur est une variable dont le contenu est une adresse (référence)
- Un pointeur NULL contient une adresse spécifique NULL
- L'opérateur d'adressage `&` permet de récupérer l'adresse d'une variable
- Un pointeur d'un type peut uniquement contenir l'adresse d'un objet du même type
- L'opérateur d'indirection `*` permet d'accéder à l'objet référencé par un pointeur
- Le type `void *` permet de construire des pointeurs génériques
- L'indicateur de conversion `%p` peut être utilisé pour afficher une adresse
- Il est possible de pointer sur un pointeur
- Il est possible de pointer sur une fonction

Les tableaux

Déclaration d'un tableau unidimentionnel

Un tableau est un espace mémoire d'adresses consécutives de variables de même type.

Syntaxe : `type identificateur[longueur];`

Exemple :

```
int tab[20]; // tableau de 20 int
int tab2[3] = { 1, 2, 3 }; // initialisation de tab2 avec les valeurs 1, 2 et 3
```

Accès aux éléments d'un tableau

L'accès aux éléments d'un tableau se réalise à l'aide d'un indice

Exemple :

```
int tab[20]; // tableau de 20 int
tab[3] = 2; // l'element 3 du tableau prend la valeur 2
int a = tab[3]; // la variable a prend la valeur du 3e element de tab
```

Nom d'un tableau et référence

Le nom d'un tableau est un pointeur constant sur le premier élément du tableau, ainsi `&tab[0]` et `tab` sont une seule et même adresse.
Exemple :

```
int tab[10];
int * ptrt;
ptrt = tab; // est equivalent a ptrt = &tab[0];
```

Parcours d'un tableau

Il est possible de parcourir un tableau en utilisant directement sa référence
Exemple :

```
int S[10] = { -3, 4, 0, -7, 3 , 8, 0, -1, 4, -9};
int T[10];
int i, j;
// Copie uniquement les elements positifs du tableau S dans T
for (i = 0, j = 0; i < 10; i++)
{
        if (*(S + i) > 0)
        {
                *(T + j) = *(S + i);
                j++;
        }
}
```

Définition d'une structure

Une structure est un regroupement de plusieurs objets, de types différents ou non.

Syntaxe : `struct etiquette { /* Objet(s) composant(s) la structure. */ };`

Exemple :

```
struct test {
        int a;                  // champ d'une variable de type int
        double * ptr;           // champ d'une variable de type pointeur double
        void (*ptrf) (int);     // champ d'une variable de type pointeur de fonction
};
```

Instatiation d'une structure

Une structure est vu comme un type.

Syntaxe : `struct etiquette identificateur;`

Exemple :

```
struct test elt; // elt est de type struct test
```

Accès à un membre d'une structure

L'accès à un membre d'une structure se fait via la variable de type structure et de l'opérateur . suivi du nom du champ visé.

Exemple :

```
elt.a;
elt.ptr;
elt.ptrf;
```

Accès à un membre d'une structure pointée

L'accès à un membre d'une structure pointée se fait via la variable de type structure et de l'opérateur -> suivi du nom du champ visé. C'est écriture est équivalente à l'utilisation de . après déférencement.

Exemple :

```
struct test elt;
struct test *elt2 = &elt;

elt2->a = 2; // equivalent a (*elt2).a = 2
```

Conversion de type explicite (*type casting*)

Principe

La conversion de type en C est le processus de conversion d'un type de données en un autre type de données par le programmeur à l'aide de l'opérateur de casting, `( type )`. Attention, lors d'une conversion, le type de données de destination peut être plus petit que le type de données source lors de la conversion du type de données en un autre type de données.

Exemple :

```
int x;
float y;
y = (float) x;
```

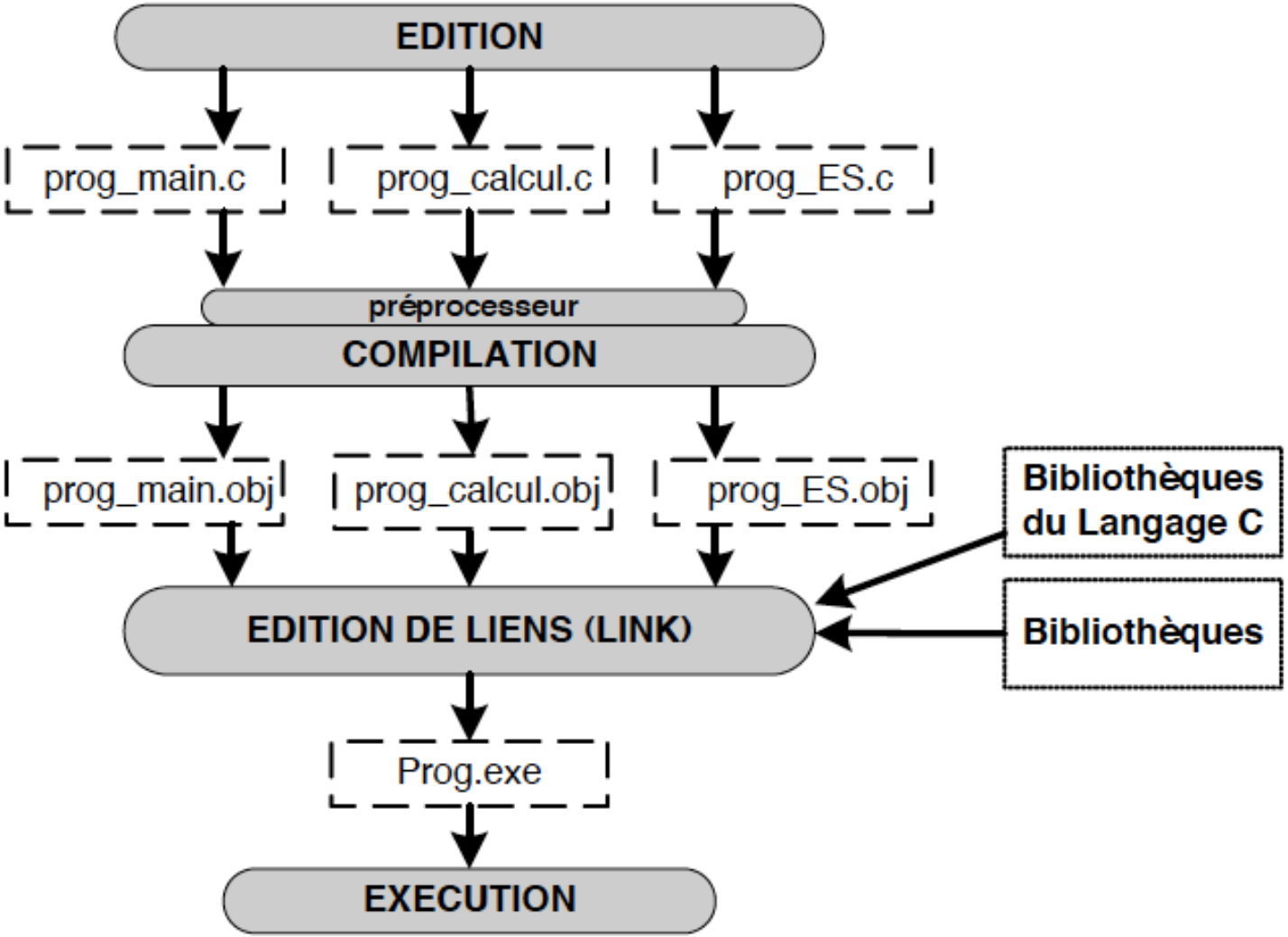
Portée des variables

Une variable n'existe que dans un bloc encadré par les accolades.

Exemple :

```
int a = 3;
{
        int b = 4;
}
a = b; // erreur car b n'existe pas dans ce bloc
```

Compilation



https ://public.iutenligne.net/informatique/algorithme-et-programmation/priou/LangageC/

Fichier d'entête

Les fichiers en-tête tels que `stdio.h`, `math.h`, etc. sont des fichiers sources qui contiennent des prototypes des fonction (ex. `printf`), des macro-instructions (ex. `getchar`), des constantes symboliques (ex. `NULL`, `M_PI`)... Ces fichiers sont utilisés par le préprocesseur pour compléter les fichiers sources : le préprocesseur remplace littéralement la directive `#include <fic.h>` par le texte contenu dans le fichier `fic.h`.

Inclusions multiples

Pour éviter qu'un fichier d'entête soit inclus plusieurs fois, on utilise une macro de protection :

```
#ifndef FOO_H
#define FOO_H
        ... contenu de foo.h
#endif /* !FOO_H */
```

On peut aussi utiliser la directive de préprocesseur `#pragma once`, mais elle n'est pas forcément prise en charge par tous les compilateurs.

La bibliothèque standard

> *La bibliothèque standard du C est une collection maintenant normalisée d'en-têtes et de routines utilisées pour implémenter des opérations courantes, telles que les entrées/sorties et la gestion des chaînes de caractères, dans le langage C. [wikipedia]*

Quelques librairies que nous allons utiliser :

- `<stdio.h>` : Fournit les capacités centrales d'entrée/sortie du langage C, comme la fonction `printf`.
- `<assert.h>` : Contient la macro `assert`, utilisée pour aider à détecter des incohérences de données et d'autres types de bogues dans les versions de débogage d'un programme.
- `<time.h>` : Pour convertir entre différents formats de date et d'heure.

Focus sur la fonction printf

Prototype :

```
int printf(const char* format, ...);
```

Une instruction de formatage peut-être utilisé ainsi avec le format précédé de %, qui sera associé au nom de la variable qu'on veut afficher.
Quelques librairies que nous allons utiliser :

Type	Lettre
int	%d
long	%ld
short	%hd
float/double	%f
char	%c
string (char*)	%s
pointeur (void*)	%p
entier hexadécimal	%x
entier octal	%o
entier non signé	%u

Exemple :

```
long nombre1 = 400, nombre2 = 500;
printf("Le nombre 1 est egal a %ld et le nombre 2 a %ld", nombre1, nombre2);
```

Gestion de la mémoire

Le langage C propose trois méthodes principales pour allouer de la mémoire aux objets :

1. **Allocation statique** : l'espace pour l'objet est alloué dans le binaire au moment de la compilation ; ces objets ont une durée de vie tant que le binaire qui les contient est chargé en mémoire.
2. **Allocation automatique** : des objets temporaires peuvent être stockés sur **la pile** (*stack*), et cet espace est automatiquement libéré et réutilisable après la sortie du bloc dans lequel ils sont déclarés.
3. **Allocation dynamique** : des blocs de mémoire de taille arbitraire peuvent être attribués dans une région de la mémoire appelée **le tas** (*heap*) pendant l'exécution à l'aide de fonctions de la bibliothèque standard (`stdlib.h`) ; ces blocs persistent jusqu'à ce qu'ils soient libérés ultérieurement pour pouvoir être réutilisés.

Allocation dynamique

Malloc : `void *malloc(size_t size)`

Alloue `size` d'octets et renvoie un pointeur sur la mémoire allouée. La mémoire n'est pas initialisée.

```
double * v = (double *)malloc(12*sizeof(double)); // reservation d'un espace memoire de 12 doubles
```

Free : `void free(void *ptr)`

Libère l'espace mémoire pointé par un `ptr`, qui doit avoir été retourné par un appel précédent à malloc() ou à des fonctions apparentées. ou d'une fonction apparentée.

```
free(v);
```

Calloc : `void *calloc(size_t nitems, size_t size)`

Alloue de la mémoire pour un tableau de `nitems` d'un octet chacun et renvoie un pointeur sur la mémoire allouée. La mémoire est mise à zéro. Renvoie un pointeur unique qui peut être passé à free().

```
int *a =  (int *)calloc(12, sizeof(int)); // allocation et mise a zero de 12 entiers de type int
```

Un peu de pratique avec le TD0

Objectifs

- Installer les outils (voir guide d'installation)
- Compiler avec gcc et exécuter un code
- Coder un petit programme en C